ESCOLA DE ENGENHARIA DE PIRACICABA
Ciência/Engenharia da Computação
Disciplina: Sistemas Distribuídos
Prof. José Martins Junior

Chat Multiusuário com Salas via Sockets TCP

Relatório técnico do trabalho em grupo
Projeto de protocolo e desenvolvimento das aplicações cliente e servidora

Integrante	RA
Gabriel de Oliveira Ramin	RA: 202310052
Gabriel Pinson	RA: 20310405
Alex Zulini Venier	RA: 202310290
João Victor de Oliveira	RA: 202310130

Piracicaba – SP
2026

Sumário

1 Introdução 3
2 Descrição do problema de comunicação de dados 4
2.1 Escolha do protocolo de transporte 4
3 Objetivos 6
3.1 Objetivo geral 6
3.2 Objetivos específicos 6
4 Requisitos da solução 7
4.1 Atendimento aos requisitos obrigatórios 7
4.2 Requisitos funcionais 7
4.3 Requisitos não funcionais 8
5 Modelo da solução 9
5.1 Arquitetura geral 9
5.2 Organização do código-fonte 9
5.3 Modelo de threads 10
5.4 Concorrência e sincronização no servidor 11
6 Projeto do protocolo 12
6.1 Transporte e enquadramento 12
6.2 Formato das mensagens 12
6.3 Byte stuffing 13
6.4 Mensagens do cliente para o servidor 14
6.5 Mensagens do servidor para o cliente 14
6.6 Códigos de erro 15
6.7 Regras de comportamento do protocolo 15
6.8 Diagramas de sequência 16
7 Comportamento das entidades 18
7.1 Entidade servidora 18
7.2 Entidade cliente – versão de terminal 19
7.3 Entidade cliente – versão gráfica 19
8 Layout das telas 21
8.1 Tela inicial 21
8.2 Painel do servidor 21
8.3 Tela de conexão 22
8.4 Tela de chat 24
8.5 Cliente de terminal 26
9 Requisitos e procedimentos de instalação 27
9.1 Requisitos 27
9.2 Instalação do JDK 27
9.3 Compilação 27
9.4 Execução 27
9.5 Problemas comuns 28
10 Procedimentos de teste 29
10.1 Ambiente utilizado 29
10.2 Casos de teste 29
10.3 Roteiro de teste para a demonstração 30
10.4 Registro de uma troca real de mensagens 30
11 Considerações finais 31
11.1 Limitações conhecidas 31
11.2 Possíveis melhorias 31
Referências 32
Apêndice A – Estrutura do projeto 33
Apêndice B – Trechos de código comentados 34

1 Introdução

Este relatório descreve o projeto e a implementação de um chat multiusuário com salas, desenvolvido como trabalho em grupo da disciplina de Sistemas Distribuídos. A solução permite que várias pessoas, em computadores diferentes de uma rede, conversem em tempo real organizadas em salas temáticas, troquem mensagens privadas e consultem o histórico recente de cada sala.

Toda a comunicação é feita exclusivamente por sockets TCP, utilizando diretamente as classes `ServerSocket` e `Socket` da linguagem Java (ORACLE, 2025a, 2025b). O grupo projetou um protocolo de aplicação textual próprio, com pares `CAMPO=valor` e byte stuffing para proteger os delimitadores, implementado no servidor e nos clientes.

A arquitetura é do tipo cliente-servidor com servidor mediador: nenhum cliente se comunica diretamente com outro. O servidor recebe cada mensagem, decide quem deve recebê-la e a repassa, além de manter os dados comuns a todos os clientes (usuários conectados, salas, participantes e histórico). Tanto o servidor quanto os clientes são multithreaded.

Foram desenvolvidas três aplicações que utilizam o mesmo protocolo e o mesmo código de comunicação:

- **Servidor**, executável pelo terminal ou por um **painel gráfico** que exibe log, usuários conectados e salas ativas;
- **Cliente de terminal**, que lê comandos digitados pelo usuário;
- **Cliente gráfico**, com tela de conexão e tela de chat com lista de salas, lista de usuários e mensagens formatadas.

Como os clientes falam o mesmo protocolo, um usuário no terminal conversa normalmente com um usuário na interface gráfica, o que evidencia que a comunicação depende apenas do formato das mensagens trocadas e não da interface utilizada.

O relatório está organizado da seguinte forma: a Seção 2 descreve o problema de comunicação resolvido; a Seção 3 apresenta os objetivos; a Seção 4 lista os requisitos e como foram atendidos; a Seção 5 detalha o modelo da solução; a Seção 6 especifica o protocolo; a Seção 7 descreve o comportamento de cada entidade; a Seção 8 apresenta o layout das telas; as Seções 9 e 10 trazem os procedimentos de instalação e de teste; e a Seção 11 apresenta as considerações finais.

2 Descrição do problema de comunicação de dados

Deseja-se permitir que diversos usuários, cada um em seu computador, troquem mensagens de texto em tempo real. As conversas acontecem em salas: quem está em uma sala recebe todas as mensagens enviadas a ela e apenas a ela. Também deve ser possível enviar uma mensagem privada diretamente a um usuário, consultar quem está na sala, quais salas existem e quais foram as últimas mensagens de uma sala.

Embora pareça simples do ponto de vista do usuário, esse cenário envolve diversos problemas típicos de sistemas distribuídos, que orientaram o projeto da solução:

Tabela 1 – Problemas de comunicação envolvidos e soluções adotadas

Problema	Descrição	Solução adotada
Localização e mediação	Os clientes não conhecem os endereços uns dos outros e podem estar atrás de redes diferentes.	Um servidor com endereço e porta conhecidos atua como ponto de encontro e mediador de todas as mensagens.
Dados comuns consistentes	Todos precisam ter a mesma visão de quem está online, quais salas existem e o que foi dito nelas.	Os dados comuns ficam somente no servidor (repositório central) e são consultados pelos clientes via protocolo.
Identificação única	Duas pessoas não podem usar o mesmo apelido, senão mensagens privadas e listas ficariam ambíguas.	Reserva de apelido atômica no servidor; em caso de conflito é devolvido o erro 02.
Concorrência no servidor	Muitos clientes enviam comandos ao mesmo tempo e acessam as mesmas estruturas de dados.	Uma thread por cliente e métodos `synchronized` nas estruturas compartilhadas.
Recebimento assíncrono	Uma mensagem pode chegar a qualquer momento, inclusive enquanto o usuário está digitando.	Cliente com duas threads: uma para entrada do usuário e envio, outra dedicada à leitura do socket.
Delimitação das mensagens	O TCP entrega um fluxo de bytes sem fronteiras entre mensagens ou campos (KUROSE; ROSS, 2021).	Uma mensagem por linha (`\n`), campos `CAMPO=valor` separados por `;`.
Delimitadores no conteúdo	O texto digitado pelo usuário pode conter `;`, `=` ou `\`, que seriam confundidos com delimitadores.	Byte stuffing: caracteres reservados são precedidos por `\` na transmissão e restaurados na recepção.
Falhas e erros	Comandos inválidos, pré-condições não atendidas e quedas de conexão sem aviso.	Respostas `ERR` com códigos numerados; detecção de desconexão e liberação de recursos no servidor.

2.1 Escolha do protocolo de transporte

O enunciado (MARTINS JUNIOR, 2026) permite o uso direto de TCP ou UDP. O grupo optou pelo TCP pelos seguintes motivos:

- **Entrega confiável e ordenada**: em um chat, uma mensagem perdida ou fora de ordem compromete a conversa. O TCP garante retransmissão e ordenação sem que a aplicação precise implementá-las (KUROSE; ROSS, 2021; EDDY, 2022).

- **Conexão persistente = sessão**: cada conexão TCP identifica naturalmente um usuário no servidor. O apelido e a sala atual ficam associados à conexão (à `ThreadCliente`), e o fim da conexão indica a saída do usuário.
- **Detecção de desconexão**: quando o cliente fecha a aplicação ou cai, a leitura do socket no servidor retorna fim de fluxo ou exceção (ORACLE, 2024a), permitindo avisar a sala e liberar o apelido.
- **Correlação simples de respostas**: como o canal é ordenado, as respostas do servidor chegam na mesma ordem dos comandos enviados, o que simplifica o protocolo (Seção 6.7).

3 Objetivos

3.1 Objetivo geral

Projetar um protocolo de aplicação (comportamento das entidades e formato das mensagens) e desenvolver, em Java, aplicações cliente e servidora multithreaded que se comuniquem exclusivamente por sockets TCP, implementando um serviço de bate-papo com salas no qual o servidor atua como mediador e repositório dos dados comuns aos clientes.

3.2 Objetivos específicos

1. Definir um **formato textual de mensagens** com pares `CAMPO=valor`, incluindo a lista de comandos, respostas, notificações e códigos de erro.
2. Implementar a técnica de **byte stuffing** para que qualquer texto digitado pelo usuário seja transmitido íntegro, sem ser confundido com os delimitadores do protocolo.
3. Implementar o tratamento das mensagens (montagem e análise) **tanto no servidor quanto nos clientes**, reutilizando a mesma classe `Protocolo`.
4. Desenvolver um **servidor multithreaded** que atenda vários clientes simultaneamente, com uma thread de atendimento por conexão.
5. Manter no servidor o **repositório de dados comuns**: apelidos em uso, salas ativas, participantes de cada sala e histórico das últimas mensagens, garantindo consistência com sincronização entre threads.
6. Desenvolver **clientes multithreaded**, capazes de receber mensagens a qualquer momento enquanto o usuário interage.
7. Oferecer as funcionalidades de identificação com apelido único, criação e troca de salas, mensagens para a sala, mensagens privadas, listagem de usuários e de salas, histórico e notificações de entrada e saída.
8. Construir interfaces de uso: **cliente de terminal**, **cliente gráfico** e **painel gráfico do servidor**.
9. Tratar erros de protocolo e **desconexões inesperadas** sem afetar os demais usuários.
10. Documentar a solução e definir **procedimentos de instalação e teste** reprodutíveis.

4 Requisitos da solução

4.1 Atendimento aos requisitos obrigatórios

A Tabela 2 relaciona cada requisito obrigatório do enunciado (MARTINS JUNIOR, 2026) com a forma como foi atendido na implementação.

Tabela 2 – Requisitos obrigatórios do enunciado (MARTINS JUNIOR, 2026) e seu atendimento

Requisito do enunciado	Como foi atendido
Pelo menos dois clientes que se comunicam através de um servidor	O servidor aceita um número ilimitado de clientes simultâneos (limitado apenas pelos recursos da máquina). Os testes foram feitos com quatro clientes ao mesmo tempo (três gráficos e um de terminal).
Servidor como entidade mediadora	Nenhum cliente conhece o endereço de outro. Toda mensagem (de sala ou privada) é enviada ao servidor, que decide os destinatários e a repassa (`Sala.difundir` e `ThreadCliente.tratarPriv`).
Servidor como repositório de dados comuns	O servidor mantém os apelidos em uso (mapa apelido → conexão), as salas ativas, os participantes de cada sala e o histórico das últimas 50 mensagens por sala. Os clientes obtêm esses dados pelos comandos `USERS`, `ROOMS` e `HIST`.
Comunicação exclusivamente via sockets, usando diretamente TCP ou UDP	Uso direto de `java.net.ServerSocket` e `java.net.Socket` (TCP). Não há HTTP, RMI, WebSocket, serialização de objetos Java ou qualquer biblioteca externa. A interface gráfica (Swing) é apenas apresentação e não participa da comunicação.
Tratamento das mensagens textuais nos clientes e no servidor	A classe `Protocolo` monta e analisa as mensagens `CAMPO=valor` com byte stuffing. Ela é usada pelo servidor (`ThreadCliente`), pelo cliente de terminal (`Cliente`, `ThreadRecebimento`) e pelo cliente gráfico (`TelaConexao`, `TelaChat`).
Utilizar e adaptar os códigos-exemplo das aulas	A estrutura segue o modelo dos exemplos: servidor com laço de `accept()` que cria uma thread de atendimento por conexão; cliente com uma thread para o teclado e outra para o recebimento. Os exemplos foram estendidos com o protocolo, as salas e as interfaces gráficas.
Clientes e servidor multithreaded	Servidor: thread principal (accept) + uma `ThreadCliente` por conexão. Cliente de terminal: thread principal (teclado) + `ThreadRecebimento`. Cliente gráfico: thread da interface (EDT) + `ThreadEscuta` + thread de conexão.
Desenvolvimento em C (gcc/Linux) ou Java	Java, opção permitida pelo enunciado (MARTINS JUNIOR, 2026): JDK 17 ou superior (testado com OpenJDK 21), sem dependências externas.

4.2 Requisitos funcionais

Tabela 3 – Requisitos funcionais

ID	Requisito	Comando do protocolo
RF01	O usuário deve se identificar com um apelido único antes de usar o chat.	`LOGIN`
RF02	O usuário deve poder entrar em uma sala; a sala é criada automaticamente se não existir.	`JOIN`

ID	Requisito	Comando do protocolo
RF03	O usuário deve poder trocar de sala, deixando de receber as mensagens da anterior.	`JOIN`
RF04	Uma mensagem enviada à sala deve ser entregue a todos os demais participantes da sala.	`MSG`
RF05	O usuário deve poder enviar uma mensagem privada a qualquer usuário conectado.	`PRIV`
RF06	O usuário deve poder listar os participantes da sala atual.	`USERS`
RF07	O usuário deve poder listar as salas ativas no servidor.	`ROOMS`
RF08	O usuário deve poder consultar as últimas N mensagens da sala.	`HIST`
RF09	Os participantes devem ser notificados quando alguém entra, sai ou se desconecta da sala.	`NOTIF`
RF10	O servidor deve informar erros com código e descrição.	`ERR`
RF11	O usuário deve poder encerrar a sessão.	`QUIT`
RF12	O administrador do servidor deve poder enviar um aviso a todos os clientes conectados.	`NOTIF`
RF13	O painel do servidor deve exibir log de eventos, usuários conectados e salas ativas.	(interno)

4.3 Requisitos não funcionais

Tabela 4 – Requisitos não funcionais

ID	Requisito
RNF01	Portabilidade: executar em Linux, Windows e macOS com JDK 17 ou superior, sem bibliotecas externas.
RNF02	Codificação: todo o texto trafega em UTF-8, permitindo acentuação.
RNF03	Concorrência segura: o acesso às estruturas compartilhadas do servidor deve ser sincronizado, sem condições de corrida nem deadlock.
RNF04	Isolamento de falhas: a queda de um cliente não pode afetar os demais.
RNF05	Responsividade: a interface gráfica não pode congelar enquanto aguarda a rede.
RNF06	Interoperabilidade: clientes de terminal e gráficos devem funcionar juntos no mesmo servidor.
RNF07	Integridade do conteúdo: qualquer texto digitado deve chegar ao destino exatamente igual (exceto quebras de linha).

5 Modelo da solução

5.1 Arquitetura geral

A solução adota a arquitetura cliente-servidor com servidor mediador (COULOURIS et al., 2013), ilustrada na Figura 1. Cada cliente mantém uma única conexão TCP com o servidor durante a sessão; por ela trafegam, nos dois sentidos, as linhas de texto do protocolo.

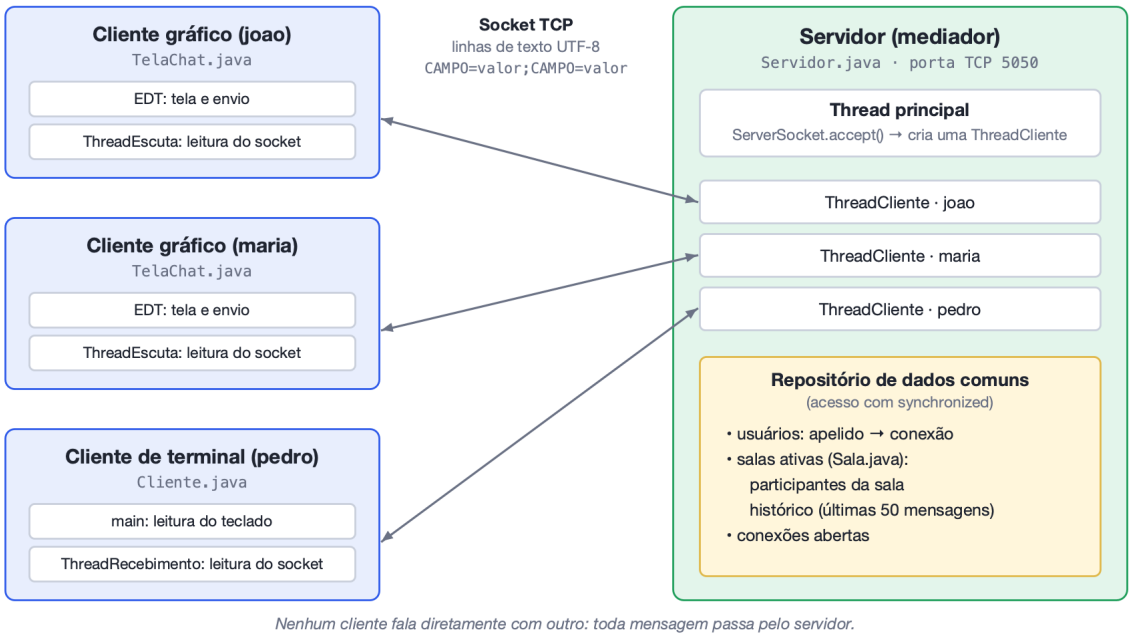


Figura 1 – Arquitetura da solução: clientes, servidor, threads e dados comuns

No servidor, a thread principal fica bloqueada em `accept()` aguardando conexões. Para cada conexão aceita é criada uma `ThreadCliente`, responsável por ler os comandos daquele cliente, executá-los e enviar as respostas. Os dados comuns (usuários, salas, participantes e históricos) ficam em objetos compartilhados por todas as threads de atendimento, com acesso sincronizado (TANENBAUM; VAN STEEN, 2007).

Quando um cliente envia uma mensagem para a sala, a sua `ThreadCliente` registra a mensagem no histórico da sala e a escreve nos sockets dos demais participantes. Assim, a comunicação entre clientes é sempre indireta: cliente → servidor → cliente(s).

5.2 Organização do código-fonte

Tabela 5 – Arquivos do código-fonte e suas responsabilidades

Arquivo	Entidade	Responsabilidade
`Protocolo.java`	Comum	Montagem e análise das mensagens `CAMPO=valor` com byte stuffing.
`Servidor.java`	Servidor	Abre o `ServerSocket`, executa o laço de `accept()`, mantém os apelidos, as conexões e as salas; oferece consultas para o painel.

Arquivo	Entidade	Responsabilidade
`ThreadCliente.java`	Servidor	Thread de atendimento de um cliente: lê e interpreta os comandos, aplica as regras e envia respostas.
`Sala.java`	Servidor	Participantes e histórico de uma sala; difusão de mensagens aos participantes.
`TelaServidor.java`	Servidor (GUI)	Painel gráfico: iniciar/parar, porta, log, tabelas de usuários e salas, aviso geral.
`Cliente.java`	Cliente terminal	Thread principal: lê o teclado, traduz comandos (`/join`, `/msg`...) em mensagens do protocolo e envia.
`ThreadRecebimento.java`	Cliente terminal	Lê o socket continuamente e exibe as mensagens recebidas de forma legível.
`ChatApp.java`	GUI	Tela inicial com as opções "Iniciar servidor" e "Entrar no chat".
`TelaConexao.java`	Cliente gráfico	Coleta IP, porta, apelido e sala; abre o socket e realiza o `LOGIN` em thread separada.
`TelaChat.java`	Cliente gráfico	Tela de chat; contém a `ThreadEscuta`, a fila de comandos pendentes e a exibição das mensagens.
`Estilo.java`	GUI	Cores, fontes e componentes visuais. Não participa da comunicação.

5.3 Modelo de threads

Servidor

- **Thread principal (ou thread "Servidor-accept" no painel gráfico)**: permanece bloqueada em `ServerSocket.accept()`. A cada conexão cria e inicia uma `ThreadCliente`, voltando imediatamente a aguardar. Assim, um cliente lento nunca impede a entrada de outros (TANENBAUM; VAN STEEN, 2007).
- **ThreadCliente (uma por conexão)**: permanece bloqueada em `readLine()` no socket do seu cliente. Ao receber uma linha, analisa com `Protocolo.analisar`, executa o comando e responde. Termina quando o cliente envia `QUIT` ou a conexão cai.
- **Painel gráfico**: a thread da interface (EDT) não executa operações bloqueantes; um `Timer` atualiza as tabelas a cada segundo e o log chega por um `Ouvinte`, repassado à EDT com `SwingUtilities.invokeLater` (ORACLE, 2024b).

Cliente de terminal

- **Thread principal**: bloqueada na leitura do teclado. Converte o que foi digitado em mensagem do protocolo e escreve no socket.
- **ThreadRecebimento** (daemon): bloqueada na leitura do socket; exibe cada mensagem recebida. Sem ela, o programa ficaria parado esperando o teclado e não mostraria as mensagens dos outros usuários.

Cliente gráfico

- **Thread de conexão**: criada ao clicar em "Entrar no chat"; abre o socket, lê a saudação e realiza o `LOGIN`, sem congelar a janela.
- **EDT (Event Dispatch Thread)**: trata cliques e teclas, envia os comandos pelo socket e atualiza os componentes da tela.
- **ThreadEscuta** (daemon): bloqueada em `readLine()`; analisa cada linha e entrega o resultado à EDT com `SwingUtilities.invokeLater`, pois componentes Swing só podem ser alterados pela EDT (ORACLE, 2024b).

5.4 Concorrência e sincronização no servidor

Como várias `ThreadCliente` executam ao mesmo tempo e acessam os mesmos objetos, foram adotadas as seguintes medidas para evitar condições de corrida e deadlock:

Tabela 6 – Pontos de sincronização no servidor

Recurso compartilhado	Risco sem sincronização	Medida adotada
Mapa de apelidos (`Servidor`)	Dois clientes pedem o mesmo apelido no mesmo instante e ambos são aceitos.	`reservarApelido` é `synchronized`: verificar e inserir formam uma operação atômica.
Mapa de salas (`Servidor`)	Duas threads criam a mesma sala em duplicidade, ou uma sala é removida enquanto alguém entra.	`obterSala` e `removerSalaSeVazia` são `synchronized`.
Participantes e histórico (`Sala`)	Lista alterada durante a difusão (ex.: alguém sai enquanto a mensagem é repassada).	Todos os métodos de `Sala` são `synchronized`.
Socket de saída de cada cliente (`ThreadCliente.enviar`)	Duas threads escrevendo ao mesmo tempo misturam caracteres de mensagens diferentes na mesma linha.	`enviar` é `synchronized` no objeto da thread do destinatário: cada linha é escrita inteira.
Resposta do `HIST`	Uma difusão de outra thread se intercala entre o `OK` e as mensagens do histórico.	O `OK` e as N mensagens são enviados dentro de um bloco `synchronized(this)`.
Ordem de bloqueios	Deadlock entre o bloqueio da `Sala` e o da `ThreadCliente`.	O histórico é copiado antes de bloquear a `ThreadCliente`; avisos gerais copiam a lista de conexões e enviam fora do bloqueio do servidor.
Campos lidos pelo painel (`apelido`, `sala`)	O painel lê valores desatualizados de outra thread.	Campos declarados `volatile`.

6 Projeto do protocolo

6.1 Transporte e enquadramento

- **Transporte**: TCP, porta padrão **5050** (configurável). A porta 5000 foi evitada porque, no macOS, é ocupada pelo serviço Receptor AirPlay.
- **Codificação**: UTF-8 nos dois sentidos.
- **Enquadramento**: cada mensagem ocupa **exatamente uma linha**, terminada por `\n`. O emissor usa `println` e o receptor, `readLine`, que devolve uma mensagem por vez, pois o TCP não preserva fronteiras entre mensagens (KUROSE; ROSS, 2021).
- **Sessão**: começa com a conexão TCP e termina com o comando `QUIT` ou com o fechamento da conexão.

6.2 Formato das mensagens

Cada mensagem é composta por pares `CAMPO=valor` separados por `;`. O primeiro campo é sempre `CMD`, que indica o tipo da mensagem:

Formato geral

```
CMD=<tipo>;CAMPO1=valor1;CAMPO2=valor2;...\n

Exemplos:
CMD=LOGIN;USER=joao
CMD=MSG;FROM=joao;TS=14:05;TEXT=Bom dia, pessoal!
CMD=ERR;COD=02;INFO=Apelido em uso: joao
```

Regras de interpretação: os nomes de campos não diferenciam maiúsculas de minúsculas (são convertidos para maiúsculas na análise); a ordem dos demais campos é livre; campos desconhecidos são ignorados; e campos ausentes assumem um valor padrão (por exemplo, `HIST` sem `N` equivale a `N=10`). Isso torna o protocolo extensível: novos campos podem ser acrescentados sem quebrar clientes antigos.

Tabela 7 – Campos utilizados no protocolo

Campo	Significado	Usado em
`CMD`	Tipo da mensagem (comando ou resposta).	Todas
`USER`	Apelido desejado.	`LOGIN`
`ROOM`	Nome da sala.	`JOIN`, `USERS` (resposta)
`TEXT`	Conteúdo da mensagem digitada.	`MSG`, `PRIV`
`TO`	Apelido do destinatário.	`PRIV`
`FROM`	Apelido do remetente (preenchido pelo servidor).	`MSG`, `PRIV` (entregues)
`TS`	Horário de recebimento no servidor (HH:mm).	`MSG`, `PRIV` (entregues)
`N`	Quantidade de mensagens do histórico.	`HIST` e sua resposta `OK`
`LIST`	Lista de nomes separados por vírgula.	`USERS`, `ROOMS` (respostas)

Campo	Significado	Usado em
`INFO`	Texto informativo para o usuário.	`OK`, `ERR`, `NOTIF`
`COD`	Código numérico do erro.	`ERR`

> **Decisão de projeto**
> O campo `FROM` é sempre preenchido **pelo servidor** a partir do apelido associado à conexão. Assim, um cliente não consegue se passar por outro usuário alterando o remetente da mensagem.

6.3 Byte stuffing

Os caracteres `;` (separador de campos), `=` (separador de valor) e `\` (escape) têm significado especial. Como o usuário pode digitar esses caracteres livremente, foi adotada a técnica de byte stuffing (inserção de caractere de escape):

1. **Na montagem** (`Protocolo.montar`), todo caractere reservado que aparece no conteúdo de um nome ou valor é precedido por `\`. Quebras de linha (`\n` e `\r`) são removidas do conteúdo, pois delimitam a mensagem.
2. **Na análise** (`Protocolo.analisar`), a linha é percorrida caractere a caractere. Ao encontrar `\`, o caractere seguinte é copiado como conteúdo literal, sem ser tratado como delimitador. Só `;` e `=` **não escapados** delimitam campos e valores.

Tabela 8 – Exemplos de byte stuffing

Conteúdo digitado	Transmitido na linha	Recuperado no destino
`ola; tudo bem? a=b`	`TEXT=ola\; tudo bem? a\=b`	`ola; tudo bem? a=b`
`C:\temp`	`TEXT=C:\\temp`	`C:\temp`
`x=1;y=2`	`TEXT=x\=1\;y\=2`	`x=1;y=2`
`ponto\; e igual\=`	`TEXT=ponto\\\; e igual\\\=`	`ponto\; e igual\=`

Sem o byte stuffing, a mensagem `ola; tudo bem? a=b` seria interpretada como três campos (`TEXT=ola`, um campo sem nome `tudo bem? a` e o valor `b`), e o destinatário receberia apenas "ola". O trecho abaixo mostra o laço de análise implementado:

Protocolo.analisar – análise com byte stuffing

```
for (int i = 0; i < linha.length(); i++) {
    char c = linha.charAt(i);
    if (escapado) {                 // caractere anterior foi '\':
        token.append(c);            // este e' conteudo literal
        escapado = false;
    } else if (c == ESCAPE) {
        escapado = true;            // proximo caractere e' literal
    } else if (c == SEP_VALOR && chave == null) {
        chave = token.toString();   // fim do nome do campo
        token.setLength(0);
    } else if (c == SEP_CAMPO) {
        gravar(campos, chave, token.toString());
        chave = null;
```

```
            token.setLength(0);
        } else {
            token.append(c);
        }
    }
    gravar(campos, chave, token.toString());
```

6.4 Mensagens do cliente para o servidor

Tabela 9 – Comandos enviados pelo cliente

Comando	Formato	Pré-condição	Respostas possíveis
LOGIN	`CMD=LOGIN;USER=<apelido>`	Ainda não identificado	`OK` ou `ERR` 02, 03, 04
JOIN	`CMD=JOIN;ROOM=<sala>`	Identificado	`OK` ou `ERR` 05, 06
MSG	`CMD=MSG;TEXT=<texto>`	Identificado e em uma sala	`OK` ou `ERR` 05, 07, 08
PRIV	`CMD=PRIV;TO=<apelido>;TEXT=<texto>`	Identificado	`OK` ou `ERR` 05, 08, 10, 11
USERS	`CMD=USERS`	Em uma sala	`USERS` ou `ERR` 07
ROOMS	`CMD=ROOMS`	Conectado	`ROOMS`
HIST	`CMD=HIST;N=<quantidade>`	Em uma sala	`OK` + N × `MSG`, ou `ERR` 07, 09
QUIT	`CMD=QUIT`	Nenhuma	Sem resposta; o servidor encerra a sessão

6.5 Mensagens do servidor para o cliente

Tabela 10 – Mensagens enviadas pelo servidor

Tipo	Formato	Quando é enviada
OK	`CMD=OK;INFO=<texto>` (no `HIST` inclui também `N=<qtd>`)	Confirmação de sucesso de `LOGIN`, `JOIN`, `MSG`, `PRIV` e `HIST`; também a saudação logo após a conexão.
ERR	`CMD=ERR;COD=<código>;INFO=<texto>`	Falha em um comando (Tabela 11).
MSG	`CMD=MSG;FROM=<apelido>;TS=<hh:mm>;TEXT=<texto>`	Mensagem de sala repassada aos demais participantes; também cada item do histórico.
PRIV	`CMD=PRIV;FROM=<apelido>;TO=<apelido>;TS=<hh:mm>;TEXT=<texto>`	Mensagem privada entregue somente ao destinatário.
NOTIF	`CMD=NOTIF;INFO=<texto>`	Alguém entrou, saiu ou se desconectou da sala; aviso do servidor para todos.
USERS	`CMD=USERS;ROOM=<sala>;LIST=<a,b,c>`	Resposta ao comando `USERS`.

Tipo	Formato	Quando é enviada
ROOMS	`CMD=ROOMS;LIST=<sala1,sala2>`	Resposta ao comando `ROOMS`.

6.6 Códigos de erro

Tabela 11 – Códigos de erro

Código	Significado	Situação típica
01	Comando desconhecido	`CMD` ausente ou com valor não previsto.
02	Apelido em uso	`LOGIN` com apelido já reservado por outra conexão.
03	Login já realizado	Segundo `LOGIN` na mesma conexão.
04	Apelido não informado	`LOGIN` sem o campo `USER` ou com valor vazio.
05	Login necessário	`JOIN`, `MSG` ou `PRIV` antes do `LOGIN`.
06	Nome da sala não informado	`JOIN` sem o campo `ROOM`.
07	Não está em nenhuma sala	`MSG`, `USERS` ou `HIST` antes de entrar em uma sala.
08	Mensagem vazia	`MSG` ou `PRIV` com `TEXT` vazio.
09	Quantidade inválida	`HIST` com `N` não numérico.
10	Destinatário não informado	`PRIV` sem o campo `TO`.
11	Usuário não encontrado	`PRIV` para apelido que não está conectado.

6.7 Regras de comportamento do protocolo

1. **Saudação**: logo após aceitar a conexão, o servidor envia `CMD=OK;INFO=Conectado...`.
2. **Uma resposta direta por comando**: cada comando recebe exatamente uma resposta direta (`OK`, `ERR`, `USERS` ou `ROOMS`), exceto `QUIT`. Como o TCP é ordenado e cada cliente é atendido por uma única thread, as respostas chegam **na mesma ordem** dos comandos. O cliente gráfico usa essa regra para associar cada resposta ao comando que a originou (Seção 7.3).
3. **Mensagens assíncronas**: `MSG`, `PRIV` e `NOTIF` podem chegar a qualquer momento, pois são originadas por ações de outros clientes. O cliente deve estar sempre lendo o socket.
4. **Histórico contíguo**: a resposta ao `HIST` é um `OK` com `N=<qtd>` seguido de exatamente `qtd` mensagens `MSG`, enviadas sem intercalação.
5. **Difusão**: uma `MSG` é gravada no histórico da sala e repassada a todos os participantes **exceto o remetente**, que recebe apenas o `OK` de confirmação.
6. **Ciclo de vida das salas**: uma sala é criada no primeiro `JOIN` com o seu nome e removida quando o último participante sai. Ao trocar de sala, o cliente sai automaticamente da anterior e os participantes dela são notificados.

7. **Encerramento**: com `QUIT` ou queda da conexão, o servidor avisa a sala (`NOTIF ... desconectou`), remove o cliente da sala, libera o apelido e fecha o socket.

6.8 Diagramas de sequência

As figuras a seguir mostram trocas de mensagens reais entre clientes e servidor. Setas contínuas representam mensagens enviadas pelos clientes; setas tracejadas representam mensagens enviadas pelo servidor.

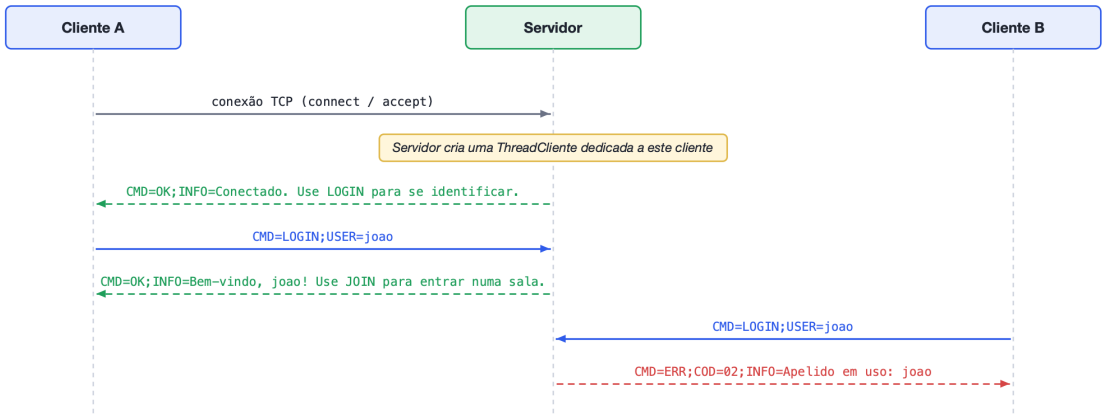


Figura 2 – Conexão, login bem-sucedido e tentativa com apelido em uso (erro 02)

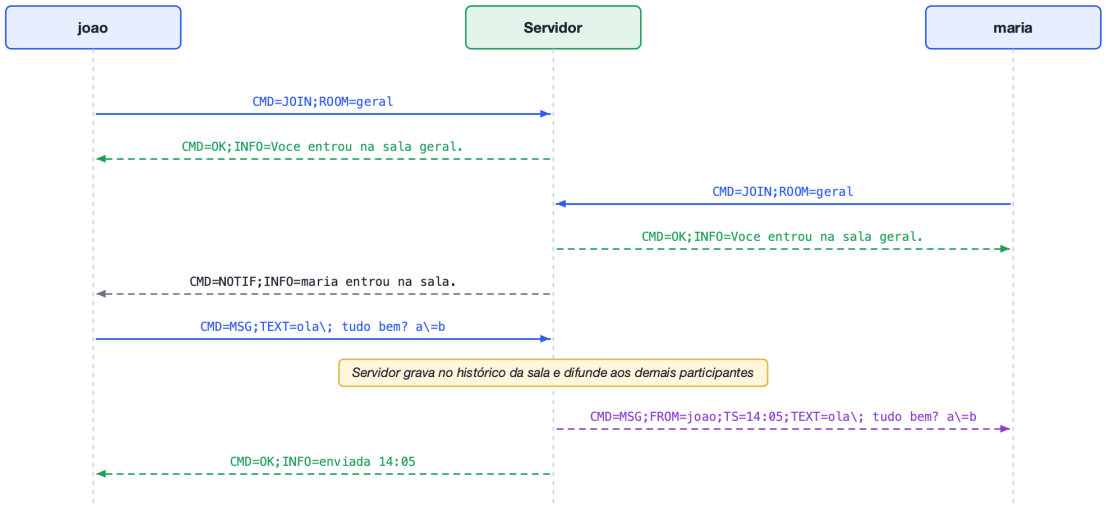


Figura 3 – Entrada na sala, notificação e mensagem com byte stuffing difundida pelo servidor

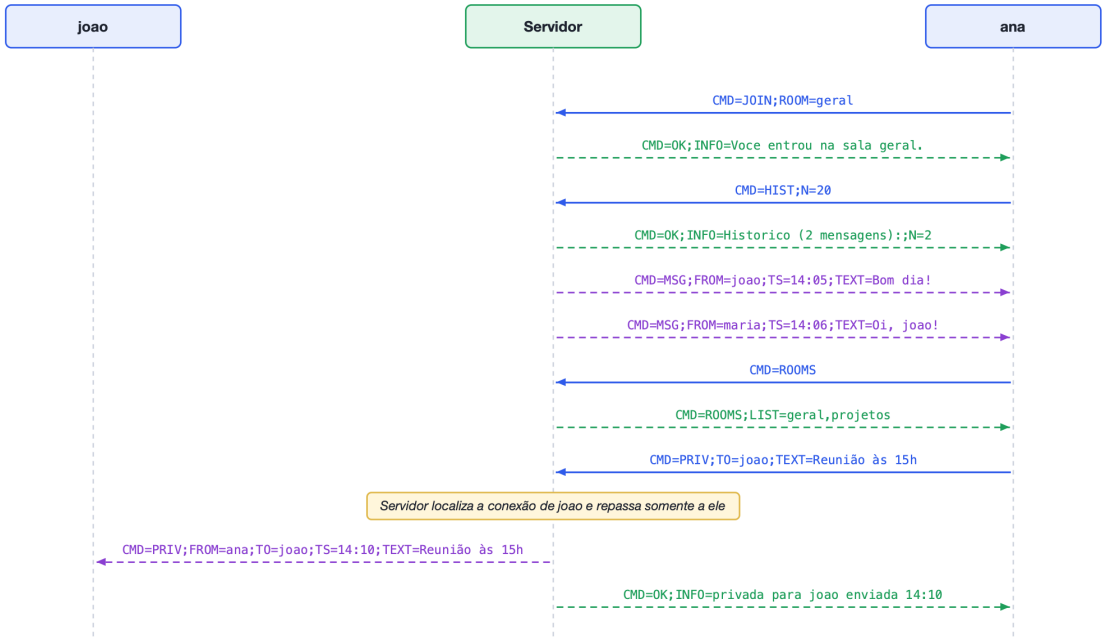


Figura 4 – Consulta ao histórico, listagem de salas e mensagem privada mediada pelo servidor

7 Comportamento das entidades

7.1 Entidade servidora

Ciclo de vida

1. Abre o `ServerSocket` na porta configurada. Se a porta estiver ocupada, informa o erro (no painel, em uma janela de diálogo).
2. Entra no laço de `accept()`. A cada conexão aceita, registra a conexão, cria uma `ThreadCliente` e a inicia (TANENBAUM; VAN STEEN, 2007).
3. Cada `ThreadCliente` envia a saudação e passa a ler linhas do socket, tratando um comando por vez.
4. Ao ser parado pelo painel, fecha o `ServerSocket` (encerrando o laço) e fecha o socket de todos os clientes, cujas threads liberam os recursos normalmente.

Estados de uma sessão

Cada conexão atendida pelo servidor passa pelos estados da Figura 5. Os comandos só são aceitos no estado adequado; fora dele, o servidor responde com o código de erro correspondente.

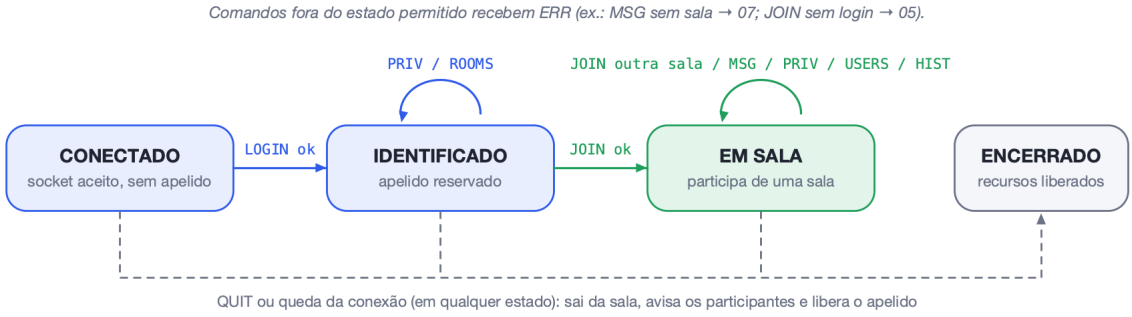


Figura 5 – Estados de uma sessão de cliente no servidor

Tratamento de cada comando

Tabela 12 – Processamento dos comandos no servidor

Comando	Ações do servidor
LOGIN	Valida o apelido; reserva-o atomicamente no mapa de usuários, associando-o à conexão; responde `OK`.
JOIN	Se já estiver em uma sala: notifica os participantes ("saiu da sala"), remove o cliente e apaga a sala se ficou vazia. Obtém ou cria a nova sala, adiciona o cliente, responde `OK` e notifica os participantes ("entrou na sala").
MSG	Gera o horário, monta `CMD=MSG;FROM;TS;TEXT`, grava no histórico (máximo de 50 mensagens por sala), difunde aos demais participantes e responde `OK` ao remetente.
PRIV	Localiza a conexão do destinatário pelo apelido; envia `CMD=PRIV` somente a ela e responde `OK` ao remetente.
USERS	Responde com os apelidos dos participantes da sala atual.

Comando	Ações do servidor
ROOMS	Responde com os nomes das salas ativas em ordem alfabética.
HIST	Copia as últimas N mensagens da sala e as envia após um `OK` com `N`, em bloco sincronizado.
QUIT / queda	Encerra o laço de leitura e executa a rotina de encerramento: notifica a sala, remove o participante, apaga a sala vazia, libera o apelido e fecha o socket.

Painel gráfico do servidor

O painel (`TelaServidor`) utiliza exatamente a mesma classe `Servidor` da versão de terminal. Ele recebe o log por meio da interface `Servidor.Ouvinte`, consulta periodicamente `resumoUsuarios()` e `resumoSalas()` para preencher as tabelas e permite enviar um aviso a todos os clientes, que é transmitido como `CMD=NOTIF`.

7.2 Entidade cliente – versão de terminal

1. Conecta ao IP e à porta informados na linha de comando.
2. Inicia a `ThreadRecebimento`, que passa a exibir tudo o que chega do servidor.
3. Na thread principal, lê cada linha digitada e a traduz em uma mensagem do protocolo (Tabela 13). Texto que não começa com `/` é enviado como `MSG`.
4. Ao digitar `/sair`, envia `QUIT`, fecha o socket e encerra.

Tabela 13 – Tradução dos comandos digitados no cliente

Digitado pelo usuário	Mensagem enviada
`/login joao`	`CMD=LOGIN;USER=joao`
`/join geral`	`CMD=JOIN;ROOM=geral`
`/users`	`CMD=USERS`
`/salas`	`CMD=ROOMS`
`/hist 5`	`CMD=HIST;N=5`
`/msg maria oi`	`CMD=PRIV;TO=maria;TEXT=oi`
`/sair`	`CMD=QUIT`
`ola; tudo bem? a=b`	`CMD=MSG;TEXT=ola\; tudo bem? a\=b`

A `ThreadRecebimento` formata cada mensagem conforme o tipo: `MSG` como `[hh:mm] autor: texto`, `PRIV` com a marca "(privado)", `NOTIF`, `USERS` e `ROOMS` com o prefixo `[*]`, `ERR` como `[ERRO código]` e `OK` como `[ok]`.

7.3 Entidade cliente – versão gráfica

Conexão e login

Na `TelaConexao`, ao clicar em "Entrar no chat", uma thread separada abre o socket (limite de 5 segundos), lê a saudação, envia `LOGIN` e aguarda a resposta. Em caso de `ERR` (por exemplo,

apelido em uso), a mensagem de erro é exibida na própria tela e o socket é fechado. Em caso de `OK`, a tela de chat é aberta recebendo o socket já autenticado.

Associação entre comandos e respostas

A tela de chat envia comandos automaticamente (por exemplo, `USERS` para atualizar a lista de participantes) e também comandos pedidos pelo usuário. Para saber a qual comando cada resposta pertence, o cliente usa a regra 2 da Seção 6.7: cada comando enviado é colocado no fim de uma fila de pendentes, e cada resposta direta recebida (`OK`, `ERR`, `USERS` ou `ROOMS`) retira o primeiro item da fila. Com isso, o cliente sabe, por exemplo, que um `OK` confirma a mensagem que acabou de ser enviada e só então a exibe na tela; ou que um `ERR` se refere a uma consulta automática e pode ser ignorado silenciosamente.

Comportamentos automáticos

- Após o `OK` de um `JOIN`: limpa a área de mensagens, atualiza o título e envia `HIST;N=20`, `USERS` e `ROOMS`, exibindo a conversa recente da sala e as listas atualizadas.
- Ao receber `NOTIF` (alguém entrou ou saiu): envia `USERS` e `ROOMS` para atualizar as listas.
- A cada 4 segundos: envia `ROOMS`, para que salas criadas por usuários de outras salas apareçam na lista.
- Ao receber `PRIV`: exibe a mensagem destacada em roxo e emite um aviso sonoro.
- Ao perder a conexão (fim do fluxo ou exceção na `ThreadEscuta`): exibe "Desconectado", limpa a lista de usuários e desabilita o envio.

8 Layout das telas

A versão gráfica foi construída com a biblioteca Swing, que acompanha o JDK (ORACLE, 2024b). As telas seguem um mesmo padrão visual: faixa escura superior com título e informações de contexto, painéis brancos com rótulos de seção e cores para diferenciar tipos de mensagem. Todas as imagens a seguir foram capturadas da aplicação em execução, com quatro usuários conectados.

8.1 Tela inicial



Figura 6 – Tela inicial (ChatApp)

Aberta com `java -cp bin ChatApp`. Contém dois cartões clicáveis: Iniciar servidor, que abre o painel do servidor, e Entrar no chat, que abre uma tela de conexão. É possível abrir várias telas de conexão, cada uma representando um usuário, o que facilita demonstrações em uma única máquina.

8.2 Painel do servidor

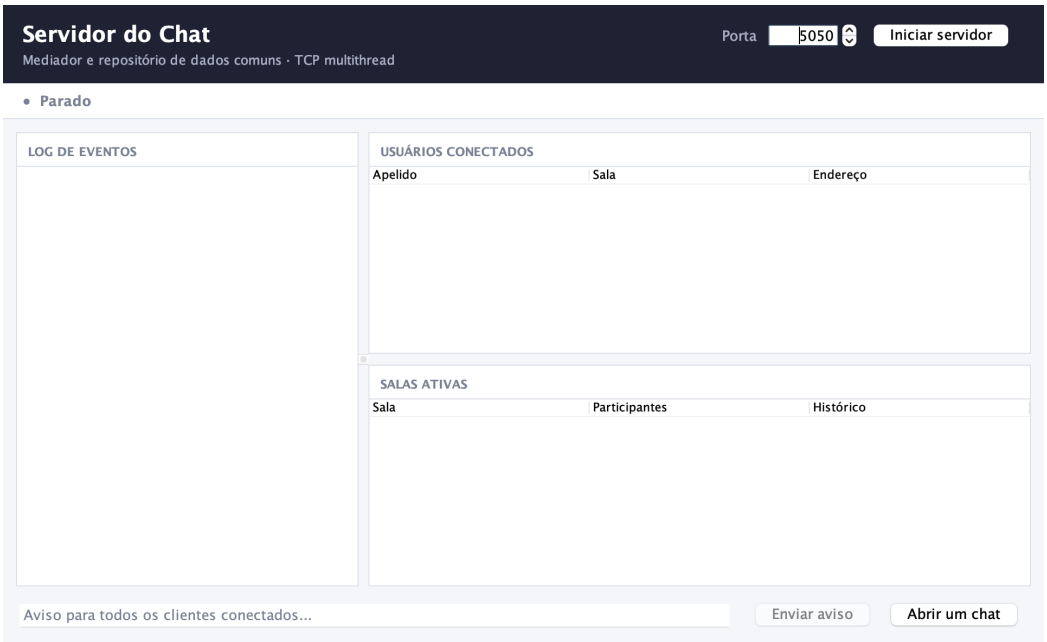


Figura 7 – Painel do servidor antes de ser iniciado

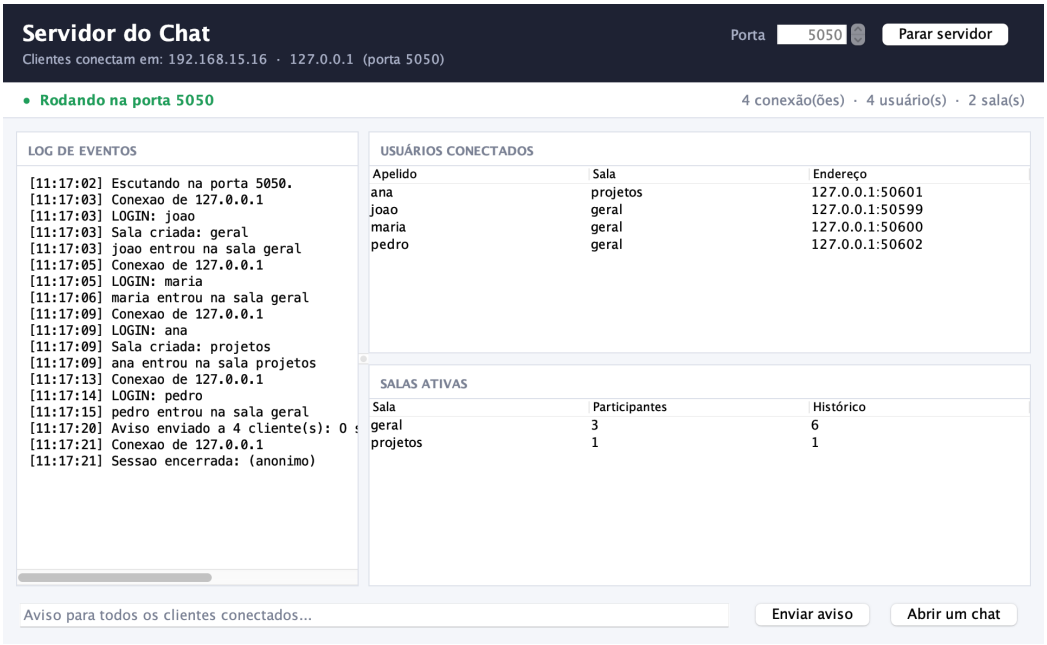


Figura 8 – Painel do servidor em execução com quatro usuários e duas salas

Tabela 14 – Áreas do painel do servidor

Área	Função
Cabeçalho	Título, campo Porta e botão Iniciar/Parar servidor. Com o servidor rodando, mostra os IPs da máquina, que devem ser informados aos clientes da rede.
Faixa de status	Indicador "Parado" ou "Rodando na porta N" e contadores de conexões, usuários e salas.
Log de eventos	Registro com horário de conexões, logins, entradas em sala, criação e remoção de salas, avisos e encerramentos.
Usuários conectados	Apelido, sala atual e endereço IP:porta de cada cliente identificado (dados comuns mantidos pelo servidor).
Salas ativas	Nome, quantidade de participantes e quantidade de mensagens no histórico de cada sala.
Rodapé	Campo e botão Enviar aviso (difundido a todos como `NOTIF`) e botão Abrir um chat, que abre uma tela de conexão já apontando para a porta do servidor.

8.3 Tela de conexão

Solicita o endereço do servidor (IP ou nome), a porta, o apelido e a sala inicial (padrão "geral"). Enquanto conecta, o botão mostra "Conectando..." e a janela continua respondendo. Erros de rede e erros do protocolo são exibidos em vermelho acima do botão, como na Figura 10, em que o apelido "maria" já estava em uso.

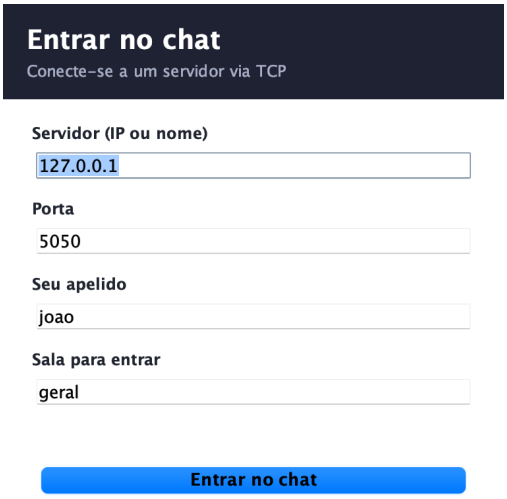


Figura 9 – Tela de conexão preenchida



Figura 10 – Tela de conexão exibindo o erro 02 retornado pelo servidor

8.4 Tela de chat

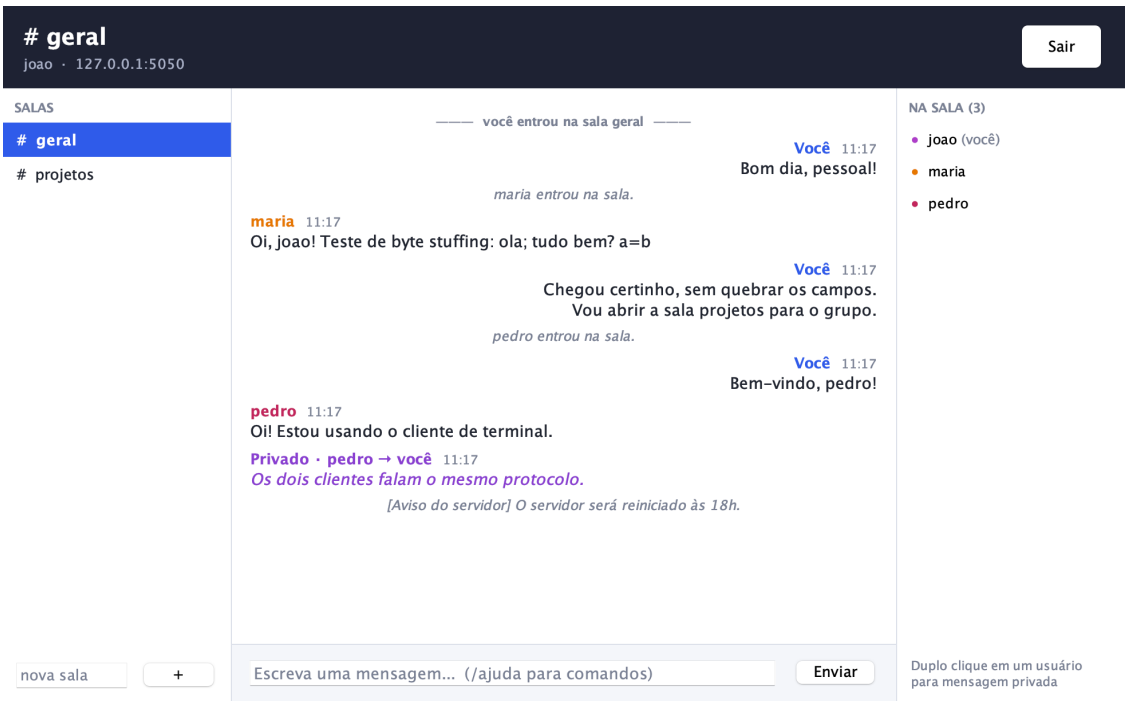


Figura 11 – Tela de chat do usuário joao na sala geral

Tabela 15 – Áreas da tela de chat

Área	Função
Cabeçalho	Sala atual (`# geral`), apelido do usuário, servidor conectado e botão Sair.
Salas (esquerda)	Salas ativas no servidor, com a sala atual destacada em azul. Um clique entra na sala; o campo "nova sala" e o botão + criam uma sala.
Mensagens (centro)	Mensagens do próprio usuário à direita, com o nome "Você" em azul; mensagens de outros à esquerda, cada autor com uma cor; notificações centralizadas em cinza; mensagens privadas em roxo; erros em vermelho; separadores para histórico e entrada na sala.
Na sala (direita)	Participantes da sala atual com contador. Duplo clique em um usuário prepara uma mensagem privada (`/msg apelido`).
Entrada (rodapé)	Campo de texto e botão Enviar. Texto comum vai para a sala; textos iniciados por `/` são comandos (`/join`, `/users`, `/salas`, `/hist`, `/msg`, `/sair`, `/ajuda`).

A Figura 12 mostra a mesma conversa do ponto de vista de maria, que entrou depois de joao: as mensagens enviadas antes de sua entrada aparecem em cinza sob "mensagens anteriores", obtidas do histórico mantido pelo servidor. Também aparecem uma mensagem privada recebida de ana, a resposta privada enviada por maria e o aviso do servidor. A Figura 13 mostra ana, na sala "projetos", que não recebe as mensagens da sala geral.

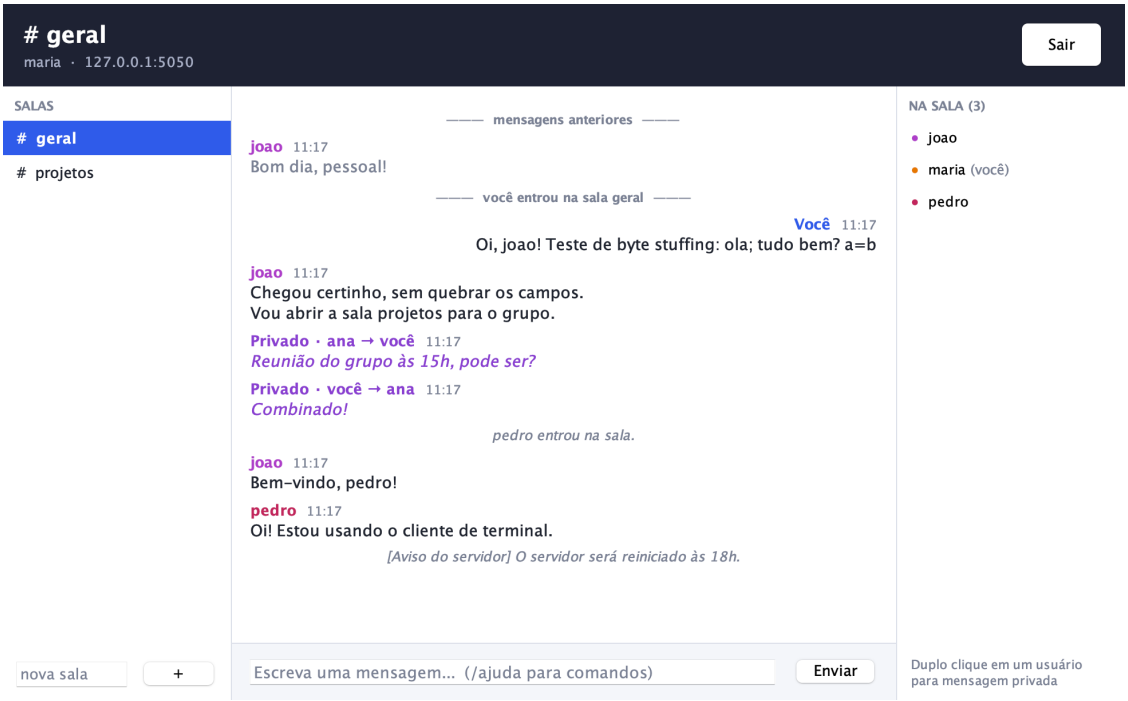


Figura 12 – Tela de chat de maria: histórico, mensagens privadas e aviso do servidor

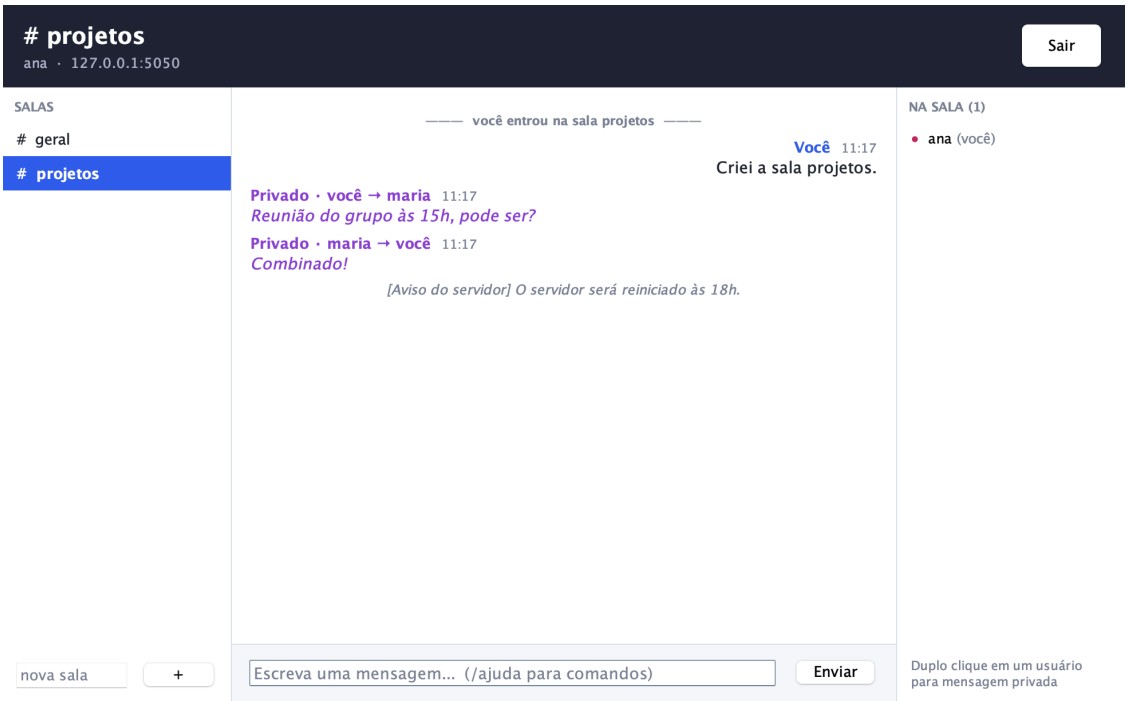


Figura 13 – Tela de chat de ana na sala projetos



Figura 14 – Tela de chat após o servidor ser parado

8.5 Cliente de terminal

A Figura 15 mostra o cliente de terminal do usuário pedro conectado ao mesmo servidor dos clientes gráficos. As linhas iniciadas por `>` são os comandos digitados. As mensagens de pedro aparecem normalmente nas telas gráficas (Figuras 11 e 12), demonstrando a interoperabilidade garantida pelo protocolo.

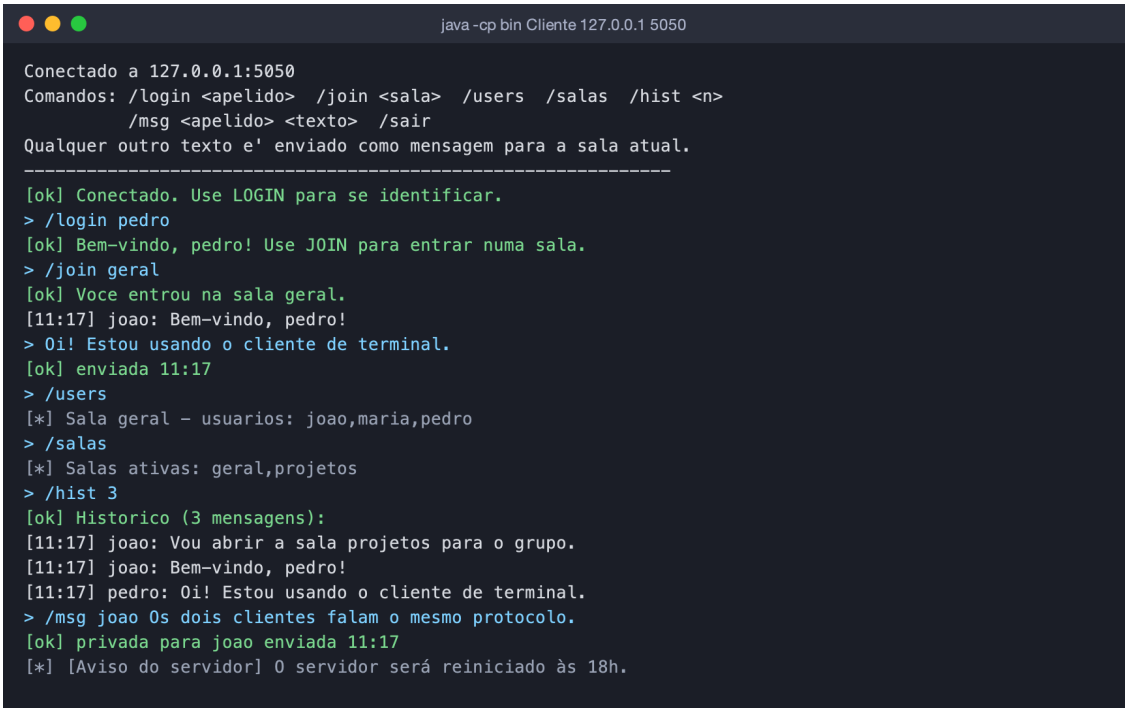


Figura 15 – Cliente de terminal conversando com os clientes gráficos

9 Requisitos e procedimentos de instalação

9.1 Requisitos

Tabela 16 – Requisitos de software e ambiente

Item	Requisito
Java	JDK 17 ou superior (testado com OpenJDK 21). É necessário o JDK (com `javac`), não apenas o JRE.
Sistema operacional	Linux, Windows ou macOS. A versão gráfica requer ambiente com interface gráfica; a versão de terminal funciona em qualquer ambiente.
Bibliotecas externas	Nenhuma. Swing, sockets e threads fazem parte do JDK.
Rede	Conectividade TCP/IP entre as máquinas (mesma rede local ou rota até o servidor).
Porta	Uma porta TCP livre no servidor (padrão 5050), liberada no firewall para conexões de entrada.
Hardware	Qualquer computador capaz de executar o JDK. Cada cliente consome uma thread e um socket no servidor.

9.2 Instalação do JDK

- **Linux (Debian/Ubuntu)**: `sudo apt install openjdk-21-jdk`
- **Windows e macOS**: instalar o Eclipse Temurin JDK 21 a partir de adoptium.net (ou, no macOS, `brew install openjdk@21`).
- Verificar a instalação com `javac -version` e `java -version`.

9.3 Compilação

Na pasta raiz do projeto (que contém a pasta `src`), executar:

```
javac -encoding UTF-8 -d bin src/*.java
```

O parâmetro `-encoding UTF-8` é necessário porque os textos das telas possuem acentuação. As classes compiladas são geradas na pasta `bin`.

9.4 Execução

Versão gráfica

```
java -cp bin ChatApp              # tela inicial
java -cp bin ChatApp servidor     # abre direto o painel do servidor
java -cp bin ChatApp cliente      # abre direto a tela de conexao
```

1. Na máquina servidora, abrir o painel, conferir a porta e clicar em **Iniciar servidor**. Anotar o IP exibido no cabeçalho.

2. Em cada máquina cliente (ou em várias janelas na mesma máquina), abrir **Entrar no chat**, informar o IP do servidor, a porta, um apelido e a sala.

Versão de terminal

```
# Terminal 1 - servidor
java -cp bin Servidor 5050

# Terminais 2, 3, ... - clientes
java -cp bin Cliente <IP-do-servidor> 5050
```

Execução em rede

- Descobrir o IP do servidor: o painel gráfico já o exibe; no terminal, usar `ip addr` (Linux), `ipconfig` (Windows) ou `ipconfig getifaddr en0` (macOS).
- Liberar a porta no firewall: no Windows, permitir o "Java(TM) Platform SE binary" em redes privadas quando solicitado; no Linux com ufw, `sudo ufw allow 5050/tcp`; no macOS, permitir conexões de entrada para o Java.
- Todas as máquinas devem estar na mesma rede. Redes de visitantes com isolamento de clientes bloqueiam a comunicação entre dispositivos.

9.5 Problemas comuns

Tabela 17 – Diagnóstico de problemas comuns

Sintoma	Causa provável	Solução
"Address already in use" / "Não foi possível abrir a porta"	Outra aplicação (ou outro servidor do chat) já usa a porta. No macOS a porta 5000 é usada pelo AirPlay.	Escolher outra porta (ex.: 5050 ou 6000) ou encerrar a aplicação que a ocupa.
"Connection refused"	Servidor não iniciado, IP errado ou porta diferente.	Iniciar o servidor e conferir IP e porta.
"Connect timed out"	Firewall bloqueando ou máquinas em redes diferentes.	Liberar a porta no firewall; testar primeiro com `127.0.0.1` na própria máquina.
"Erro 02: Apelido em uso"	O apelido já está em uso por outro cliente conectado.	Escolher outro apelido.
Acentos incorretos no terminal do Windows	Console fora de UTF-8.	Executar `chcp 65001` antes de iniciar o cliente ou usar a versão gráfica.
Erro de compilação com caracteres estranhos	Compilação sem `-encoding UTF-8`.	Compilar com o comando da Seção 9.3.

10 Procedimentos de teste

10.1 Ambiente utilizado

Os testes foram executados em macOS com OpenJDK 21.0.12, com servidor e clientes na mesma máquina (endereço `127.0.0.1`), utilizando quatro clientes simultâneos: três gráficos e um de terminal. Os testes do protocolo foram feitos também com um cliente de teste que envia linhas brutas pelo socket, para verificar individualmente cada resposta e cada código de erro. Recomenda-se repetir o roteiro da Seção 10.3 com máquinas diferentes na rede do laboratório antes da demonstração.

10.2 Casos de teste

Tabela 18 – Casos de teste executados

ID	Objetivo	Procedimento	Resultado esperado	Result.
T01	Conexão e saudação	Conectar um cliente ao servidor.	`OK` "Conectado. Use LOGIN..."	Aprovado
T02	Login válido	`/login joao`	`OK` "Bem-vindo, joao!"; aparece no painel.	Aprovado
T03	Apelido duplicado	Segundo cliente faz login como joao.	`ERR` 02; o primeiro não é afetado.	Aprovado
T04	Pré-condições	`JOIN` antes do login; `MSG`/`USERS` sem sala; segundo `LOGIN`.	`ERR` 05, 07 e 03.	Aprovado
T05	Validação de campos	`LOGIN` sem `USER`; `JOIN` sem `ROOM`; `MSG` vazia; `HIST;N=abc`; `PRIV` sem `TO`; `CMD=XYZ`.	`ERR` 04, 06, 08, 09, 10 e 01.	Aprovado
T06	Entrada em sala e notificação	joao e maria fazem `/join geral`.	Quem já estava recebe "maria entrou na sala."	Aprovado
T07	Mensagem na sala	joao envia "Bom dia, pessoal!".	maria recebe `MSG` com `FROM=joao`; joao recebe `OK`.	Aprovado
T08	Byte stuffing	Enviar `ola; tudo bem? a=b` e `barra \ e ponto; e igual=`.	Texto chega idêntico, sem quebrar campos.	Aprovado
T09	Listagens	`/users` e `/salas`.	Lista dos participantes e das salas ativas.	Aprovado
T10	Histórico	Novo cliente entra na sala e usa `/hist 3`.	`OK` com `N` seguido das últimas mensagens.	Aprovado
T11	Isolamento entre salas	joao vai para a sala "outra"; maria envia na "geral".	joao não recebe; maria vê "joao saiu da sala."	Aprovado
T12	Mensagem privada	`/msg maria Reunião às 15h` a partir de outra sala.	Só maria recebe `PRIV`; remetente recebe `OK`.	Aprovado
T13	Privada para inexistente	`/msg ninguem oi`	`ERR` 11.	Aprovado
T14	Saída e remoção de sala	Único participante de uma sala envia `/sair`.	Demais recebem "desconectou"; sala some de `ROOMS`.	Aprovado
T15	Interoperabilidade	Cliente de terminal e clientes gráficos na mesma sala.	Todos trocam mensagens normalmente.	Aprovado

ID	Objetivo	Procedimento	Resultado esperado	Result.
T16	Aviso do servidor	Enviar aviso pelo painel.	Todos os clientes recebem `NOTIF`.	Aprovado
T17	Parada do servidor	Clicar em "Parar servidor".	Clientes exibem "Desconectado".	Aprovado
T18	Porta ocupada	Iniciar o servidor em porta já em uso.	Mensagem de erro, sem travar.	Aprovado
T19	Execução em rede	Servidor e clientes em máquinas diferentes.	Mesmo comportamento de T06 a T15.	Realizar no laboratório

10.3 Roteiro de teste para a demonstração

1. Compilar o projeto e abrir `java -cp bin ChatApp` na máquina servidora; clicar em **Iniciar servidor**.
2. Abrir dois clientes (joao e maria) na sala geral. Conferir as tabelas do painel.
3. Tentar entrar novamente como maria: a tela de conexão deve mostrar o **erro 02**.
4. Trocar mensagens, incluindo `ola; tudo bem? a=b` (**byte stuffing**).
5. Abrir um terceiro cliente na sala geral: ele deve ver as **mensagens anteriores**.
6. Criar a sala "projetos" em um cliente: ela aparece na lista dos demais; mensagens da geral não chegam a ele.
7. Dar duplo clique em um usuário e enviar uma **mensagem privada**.
8. Conectar um **cliente de terminal** e conversar com os clientes gráficos; usar `/users`, `/salas` e `/hist 3`.
9. Enviar um **aviso** pelo painel do servidor.
10. Fechar uma janela de chat (notificação "desconectou") e, por fim, **parar o servidor**.

10.4 Registro de uma troca real de mensagens

O registro abaixo foi obtido com um cliente de teste enviando linhas brutas ao servidor. As linhas marcadas com `>>` foram enviadas pelo cliente e as marcadas com `<<` foram recebidas.

```
a << CMD=OK;INFO=Conectado. Use LOGIN para se identificar.
a >> CMD=LOGIN;USER=joao
a << CMD=OK;INFO=Bem-vindo, joao! Use JOIN para entrar numa sala.
a >> CMD=JOIN;ROOM=geral
a << CMD=OK;INFO=Voce entrou na sala geral.
a >> CMD=MSG;TEXT=barra \\ e ponto\; e igual\=
a << CMD=OK;INFO=enviada 11:19
b >> CMD=LOGIN;USER=maria
b << CMD=OK;INFO=Bem-vindo, maria! Use JOIN para entrar numa sala.
b >> CMD=JOIN;ROOM=geral
b << CMD=OK;INFO=Voce entrou na sala geral.
a << CMD=NOTIF;INFO=maria entrou na sala.
b >> CMD=HIST;N=5
b << CMD=OK;INFO=Historico (1 mensagens):;N=1
b << CMD=MSG;FROM=joao;TS=11:19;TEXT=barra \\ e ponto\; e igual\=
a >> CMD=JOIN;ROOM=outra
a << CMD=OK;INFO=Voce entrou na sala outra.
b << CMD=NOTIF;INFO=joao saiu da sala.
```

11 Considerações finais

O trabalho resultou em um sistema de chat funcional que atende a todos os requisitos obrigatórios: dois ou mais clientes se comunicam exclusivamente por sockets TCP através de um servidor que atua como mediador e repositório dos dados comuns; o tratamento das mensagens textuais, com pares campo=valor e byte stuffing, está implementado nos clientes e no servidor; e todas as aplicações são multithreaded e desenvolvidas em Java.

O desenvolvimento evidenciou, na prática, conceitos centrais de sistemas distribuídos: a necessidade de enquadramento das mensagens sobre um fluxo de bytes; a importância de proteger os delimitadores com byte stuffing; o papel do servidor como ponto único de coordenação e de consistência dos dados; o uso de threads para atender clientes simultâneos e receber mensagens de forma assíncrona; e os cuidados de sincronização para evitar condições de corrida e deadlock.

11.1 Limitações conhecidas

- O histórico é mantido apenas em memória: é perdido quando o servidor é reiniciado ou quando a sala fica vazia e é removida.
- A comunicação não é criptografada nem há autenticação com senha; o apelido identifica o usuário apenas durante a sessão.
- As listas `LIST` usam vírgula como separador; apelidos ou nomes de sala contendo vírgula aparecem divididos nas listas.
- O modelo de uma thread por cliente é adequado para dezenas ou centenas de usuários, mas não para milhares de conexões simultâneas.

11.2 Possíveis melhorias

- Persistir salas e históricos em arquivo, permitindo recuperação após reinício.
- Adicionar autenticação com senha e criptografia do canal (por exemplo, `SSLSocket`, que continua sendo comunicação via sockets TCP).
- Utilizar um pool de threads (`ExecutorService`) para limitar o consumo de recursos no servidor.
- Escapar a vírgula nas listas ou restringir os caracteres permitidos em apelidos e nomes de sala.
- Descoberta automática do servidor na rede local por difusão UDP.

Referências

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

EDDY, W. (ed.). **RFC 9293**: Transmission Control Protocol (TCP). [S. l.]: IETF, ago. 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9293. Acesso em: 5 nov. 2026.

KUROSE, J. F.; ROSS, K. W. **Redes de computadores e a Internet**: uma abordagem top-down. 8. ed. São Paulo: Pearson, 2021.

MARTINS JUNIOR, J. **Especificação do trabalho em grupos – Sistemas Distribuídos**. Piracicaba: Escola de Engenharia de Piracicaba, 2026.

ORACLE. **The Java Tutorials**: All About Sockets. Redwood Shores: Oracle, 2024a. Disponível em: https://docs.oracle.com/javase/tutorial/networking/sockets/index.html. Acesso em: 5 nov. 2026.

_____. **The Java Tutorials**: Concurrency in Swing. Redwood Shores: Oracle, 2024b. Disponível em: https://docs.oracle.com/javase/tutorial/uiswing/concurrency/index.html. Acesso em: 5 nov. 2026.

_____. **Java SE 21 & JDK 21 API Specification**: class ServerSocket. Redwood Shores: Oracle, 2025a. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/ServerSocket.html. Acesso em: 5 nov. 2026.

_____. **Java SE 21 & JDK 21 API Specification**: class Socket. Redwood Shores: Oracle, 2025b. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/Socket.html. Acesso em: 5 nov. 2026.

TANENBAUM, A. S.; VAN STEEN, M. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

Apêndice A – Estrutura do projeto

```
SistemaDistribuidos/
|-- src/
|   |-- Protocolo.java           tratamento das mensagens (byte stuffing)
|   |-- Servidor.java            servidor: accept, apelidos, salas
|   |-- ThreadCliente.java       thread de atendimento (uma por cliente)
|   |-- Sala.java                participantes + historico
|   |-- Cliente.java             cliente de terminal (thread de teclado)
|   |-- ThreadRecebimento.java   cliente de terminal (thread de escuta)
|   |-- ChatApp.java             tela inicial
|   |-- TelaServidor.java        painel grafico do servidor
|   |-- TelaConexao.java         tela de conexao e login
|   |-- TelaChat.java            tela de chat (contem a ThreadEscuta)
|   |-- Estilo.java              aparencia das telas
|   '-- LEIAME.txt               instrucoes resumidas
'-- bin/                         classes compiladas (javac -d bin)
```

Apêndice B – Trechos de código comentados

B.1 Laço de aceitação de conexões do servidor. A thread principal só aceita conexões e delega o atendimento, de modo que o servidor atende vários clientes ao mesmo tempo.

Servidor.java

```
public void aceitarConexoes() {
    try {
        while (rodando) {
            Socket socket = servidor.accept();   // bloqueia ate chegar conexao
            log("Conexao de " + socket.getInetAddress().getHostAddress());
            ThreadCliente t = new ThreadCliente(socket, this);
            t.start();                            // atendimento em thread propria
        }
    } catch (IOException e) {
        if (rodando) log("Erro: " + e.getMessage());
    } finally {
        parar();
    }
}
```

B.2 Interpretação dos comandos na thread de atendimento.

ThreadCliente.java

```
String linha;
while ((linha = entrada.readLine()) != null) {   // bloqueia ate chegar mensagem
    Map<String, String> campos = Protocolo.analisar(linha);
    String cmd = Protocolo.campo(campos, "CMD", "").toUpperCase();

    if (cmd.equals("LOGIN"))       tratarLogin(campos);
    else if (cmd.equals("JOIN"))   tratarJoin(campos);
    else if (cmd.equals("MSG"))    tratarMsg(campos);
    else if (cmd.equals("PRIV"))   tratarPriv(campos);
    else if (cmd.equals("USERS"))  tratarUsers();
    else if (cmd.equals("ROOMS"))  tratarRooms();
    else if (cmd.equals("HIST"))   tratarHist(campos);
    else if (cmd.equals("QUIT"))   break;
    else erro("01", "Comando desconhecido: " + cmd);
}
```

B.3 Mediação: gravação no histórico e difusão para a sala.

ThreadCliente.java e Sala.java

```
// ThreadCliente.tratarMsg
String hora = new SimpleDateFormat("HH:mm").format(new Date());
String difusao = Protocolo.montar("CMD", "MSG", "FROM", apelido, "TS", hora, "TEXT",
texto);
sala.registrarHistorico(difusao);   // repositorio de dados comuns
sala.difundir(difusao, this);       // mediacao: o servidor repassa
enviar(Protocolo.montar("CMD", "OK", "INFO", "enviada " + hora));

// Sala.difundir
public synchronized void difundir(String linha, ThreadCliente remetente) {
    for (ThreadCliente c : participantes) {
        if (c != remetente) c.enviar(linha);
    }
}
```

B.4 Thread de escuta do cliente gráfico. A leitura bloqueante fica fora da thread da interface, e as atualizações da tela são entregues à EDT.

TelaChat.java

```
private class ThreadEscuta extends Thread {
    @Override
    public void run() {
        String motivo = "conexão encerrada pelo servidor";
        try {
            String linha;
            while ((linha = entrada.readLine()) != null) {
                final Map<String, String> campos = Protocolo.analisar(linha);
                SwingUtilities.invokeLater(() -> tratar(campos));
            }
        } catch (IOException e) {
            motivo = e.getMessage();
        }
        final String m = motivo;
        SwingUtilities.invokeLater(() -> conexaoPerdida(m));
    }
}
```